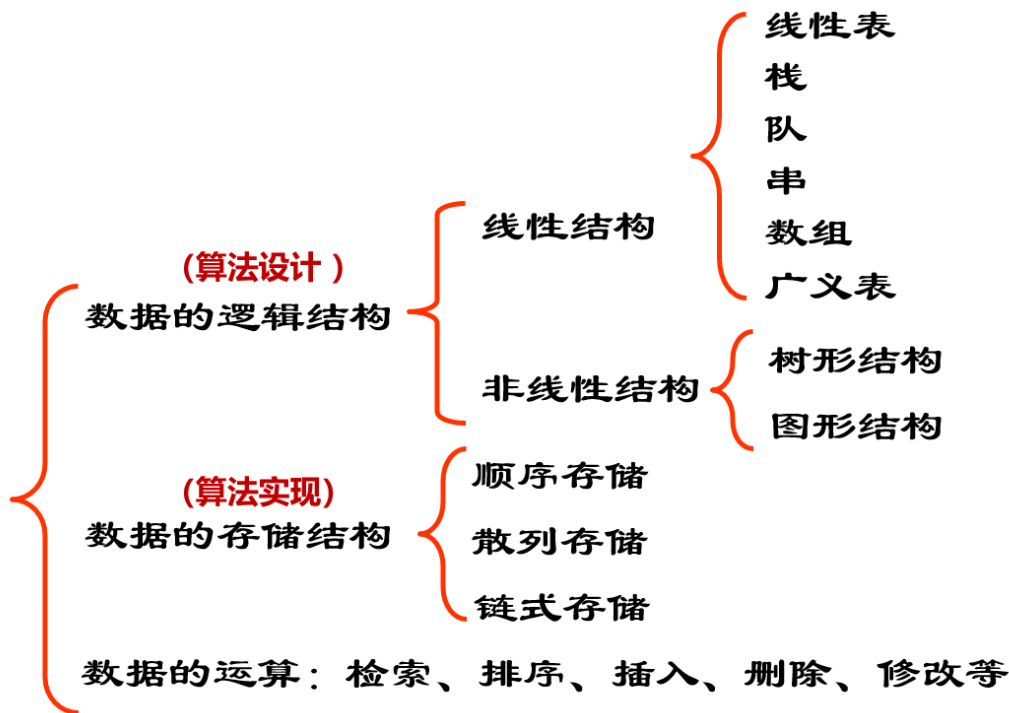


数据结构

1 绪论

1.1 数据结构总览



1.2 时间空间复杂度的计算

2 线性表

2.1 线性表基本结构及操作

```
1 ADT LinearList{
2     数据元素: D = { ai | ai ∈ D0, i=1,2,..., n    n ≥ 0 , D0为某一数据对象 }
3     关系:      S = { <ai, ai+1> | ai, ai+1 ∈ D0, i=1,2, ...,n-1 }
4     基本操作:
5     1) InitList(L)      操作前提: L为未初始化线性表。操作结果: 将L初始化为空表。
6     2) DestroyList(L)   操作前提: 线性表L已存在。 操作结果: 将L销毁。
7     3) ClearList(L)     操作前提: 线性表L已存在 。操作结果: 将表L置为空表。
8     .....
9 }ADT LinearList
```

2.2 顺序表结构及操作

2.2.1 结构

```
1  #define MAXSIZE 100                                /*线性表可能达到的最大长度*/
2  typedef struct
3  {
4      ElemType elem[MAXSIZE];                        /*线性表占用的数组空间*/
5      int last;
6      /*记录线性表中最后一个元素在数组elem[ ]中的位置(下标值),空表置为-1。注意,数组中实际
       存储的元素个数为 last+1 */
7  } SeqList;
8      /*注意区分元素的序号和数组的下标,如a1的序号为1,而其对应的数组下标为0。*/
```

2.2.2 查找

```
1  int Locate(SeqList L, ElemType e) {
2      i=0 ;                                           /*i为扫描计数器,初值为0,即从第一个元素开始比较*/
3      while ((i<=L.last)&&(L.elem[i]!=e) )
4          i++;                                       /*顺序扫描表,直到找到值为e的元素,或扫描到表尾而没找到
       */
5      if (i<=L.last)
6          return(i);                               /*若找到值为e的元素,则返回其序号*/
7      else
8          return(-1);                               /*若没找到,则返回空序号*/
9  }
10 /*算法的时间复杂度为O(n)*/
```

2.2.3 插入

```
1  #define OK 1
2  #define ERROR 0
3  int InsList(SeqList *L,int i,ElemType e) {
4      int k;
5      if( (i<1) || (i>L->last+2) ) {               /*首先判断插入位置是否合法*/
6          printf("插入位置i值不合法");
7          return(ERROR);    }
8      if( L->last>=MAXSIZE-1) {
9          printf("表已满无法插入");
10         return(ERROR);    }
11     for( k=L->last;k>=i-1;k--)                     /*为插入元素而移动位置*/
12         L->elem[k+1]=L->elem[k];
13     L->elem[i-1]=e;                                /*在C语言中数组第i个元素的下标为i-1*/
14     L->last++;
15     return(OK);
16 }
```

插入的本质就是将该位置往后的所有数据向后平移一格。

数据结构的题目中很重要的一点就是**边界**判断,包括**数据**是否合法,**位置**是否合法等。

当对结构进行更改时还要注意对**参数**更改,例如**线性表的长度参数**。

2.2.4 删除

```
1  /*在顺序表L中删除第i个数据元素，并用指针参数e返回其值*/
2  int DelList(SeqList *L,int i, ElemType *e) {
3      int k;
4      if((i<1) || (i>L->last+1)) {
5          printf("删除位置不合法！");
6          return(ERROR);
7      }
8      *e = L->elem[i-1];          /* 将删除的元素存放到e所指向的变量中*/
9      for(k=i;k<=L->last;k++)
10         L->elem[k-1]= L->elem[k];    /*将后面的元素依次前移*/
11     L->last--;
12     return(OK);
13 }
```

同样的,在删除时,本质上就是将后面的数据向前平移一格。

这里也不能忘记删除边界判断和更改**长度参数**!

2.2.5 合并*

```
1  void merge(SeqList *LA, SeqList *LB, SeqList *LC){
2      int i,j,k,l;
3      i=0;j=0;k=0;
4      while(i<=LA->last&& j<=LB->last)    /* 只要有一个表被遍历了就结束循环 */
5          if(LA->elem[i]<=LB->elem[j]) {
6              LC->elem[k]= LA->elem[i];
7              i++; k++; }
8          else {
9              LC->elem[k]=LB->elem[j];
10             j++; k++; }
11     while(i<=LA->last) {                  /* 当表LA有剩余元素，则将余下的元素赋给表LC
12         LC->elem[k]= LA->elem[i];
13         i++; k++; }
14     while(j<=LB->last) {                  /* 当表LB有剩余元素，则将余下的元素赋给表LC
15         LC->elem[k]= LB->elem[j];
16         j++; k++; }
17     LC->last=LA->last+LB->last+1;
18 }
19
```

这里有几个注意点:

1. 只用遍历完一个表,把另一个表剩下的部分直接接在后面就OK.
2. i,j,k三个参数分别表示在三个表中的位置,结束时要记得更新LC长度参数为k.

2.3 线性表的链式储存

2.3.1 链表的结构

```
1 typedef struct Node {           /* 结点类型定义 */
2     ElemType data;
3     struct Node *next;
4 }Node, *LinkedList;             /* LinkedList为结构指针类型*/
```

LinkedList 和 **Node** 同为结构指针类型

LinkedList 一般用作单链表的头指针变量

Node 一般用作指向单链表中结点的指针

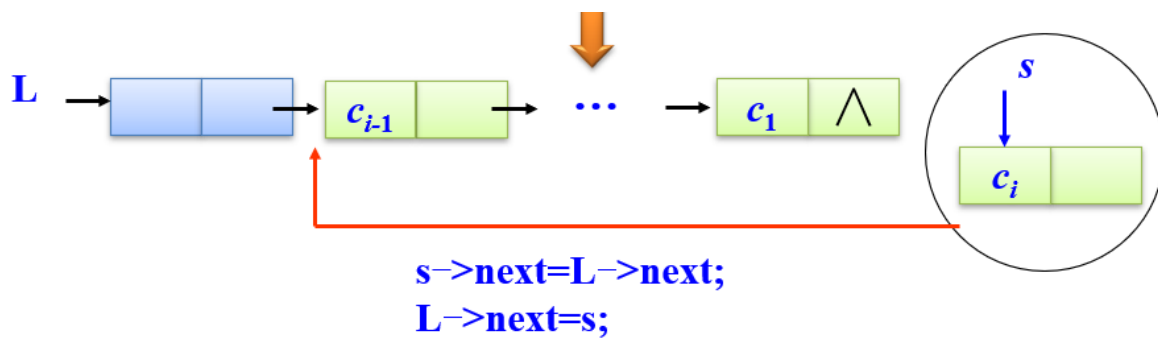
```
1 *L = (LinkedList)malloc(sizeof(Node));
```

上方代码是一个申请空节点的操作,注意**malloc**函数的运用,L是**LinkedList**类型的指针,大小为**sizeof(node)**.

一般使用的时候指针变量类型都是**LinkedList**.

2.3.2 头插法创建链表

```
1 void CreateFromHead(LinkedList L) {
2     Node *s;
3     char c;
4     int flag=1;
5     while(flag) {                               /* flag初值为1,当输入"$"时,置flag为0,建表结
束*/
6         c=getchar();
7         if(c!='$') {
8             s=(Node*)malloc(sizeof(Node)); /*建立新结点s*/
9             s->data=c;
10            s->next=L->next;                  /*将s结点插入表头*/
11            L->next=s;
12        }
13        else
14            flag=0;
15    }
16 }
```

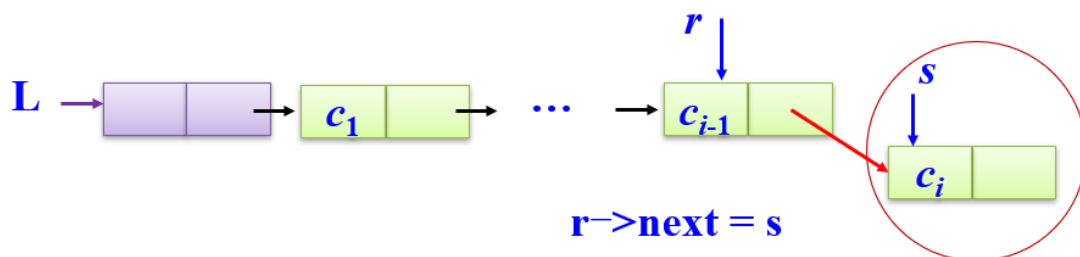


2.3.3 尾插法创建链表

```

1 void CreateFromTail(LinkList L){ /* flag初值为1, 当输入"$"时, flag为0, 建表结束
   */
2     Node *r, *s;
3     char c;
4     int flag = 1;
5     r=L; /* r 指针动态指向链表的当前表尾*/
6     while(flag)
7         c=getchar();
8         if(c!='$') {
9             s=(Node*)malloc(sizeof(Node));
10            s->data=c;
11            r->next=s;
12            r=s;
13        }
14        else {
15            flag=0;
16            r->next=NULL; /*将最后一个结点的next链域置为空, 表示链表的结束*/
17        }
18    }
19 }

```



2.3.4 头插法与尾插法的比较

头插法是锁定一个**尾节点**,在**尾节点的前面**挨个插入,因此生成的是**倒序**.

尾插法是锁定一个**头结点**,在头结点的**后面**挨个插入,因此生成的是**顺序**.

2.4 循环链表

```
1 void initClinklist(LinkList *CL) { /* CL是循环单链表的头指针变量 */
2     *CL = (LinkList)malloc(sizeof(Node)); /* 建立头结点 */
3     (*CL) -> next = *CL ; /*置为空表*/
4 }
5 void CreateClinklist (LinkList CL){/* CL已经初始化，当输入$时，建表结束 */
6     Node *rear, *s;
7     char c;
8     rear = CL; /* rear 指针动态指向链表的当前表尾 */
9     c=getchar(); /* 读入第一个元素 */
10    while(c!='$') {
11        s = (Node*)malloc(sizeof(Node));
12        s->data = c;
13        rear->next = s;
14        rear = s;
15        c=getchar();
16    }
17    rear->next = CL; /*将最后一个结点的next链域指向头结点*/
18 }
```

3 栈和队列

3.1 栈

3.1.1 栈的基本结构

```
1 #define Stack_Size 50
2 typedef struct{
3     StackElementType elem[Stack_Size]; /*用来存放栈中元素的一维数组*/
4     int top; /*用来存放栈顶元素的下标，top为-1表示空栈*/
5 }SeqStack;
```

栈是一种只能在一端进行插入或删除操作的线性表。

栈只能选取同一个端点进行插入和删除操作

3.1.2 栈的基本操作

```
1 int IsEmpty(SeqStack *S) { /*判栈S为空栈时返回值为真，反之为假*/
2     return(S->top==-1?TRUE:FALSE);
3 }
4
```

```

5  int IsFull(SeqStack *S) {    /*判栈S为满时返回真，否则返回假*/
6      return(S->top== Stack_Size-1? TRUE: FALSE);
7  }
8  int Push(SeqStack * S, StackElementType x) {
9      if(S->top== Stack_Size-1)
10         return(FALSE); /*栈已满*/
11     S->top++;
12     S->elem[S->top]=x;
13     return(TRUE);
14 }
15 int Pop(SeqStack *S, StackElementType *x){
16     if(S->top== -1)          /*栈为空*/
17         return(FALSE);
18     else {
19         *x= S->elem[S->top];
20         S->top--;          /* 修改栈顶指针 */
21         return(TRUE);
22     }
23 }
24 int GetTop (SeqStack *S, StackElementType *x) {
25     /* 将栈S的栈顶元素弹出，放到x所指的存储空间中，但栈顶指针保持不变 */
26     if(S->top== -1)          /*栈为空*/
27         return(FALSE);
28     else {
29         *x = S->elem[S->top];
30         return(TRUE);
31     }
32 }

```

3.2 队列

3.2.1 队列的基本定义

队列只能选取一个端点进行插入操作，另一个端点进行删除操作

把进行插入的一端称做**队尾**（rear）。

进行删除的一端称做**队首**（front）。

向队列中插入新元素称为**入队**，新元素入队后就成为**新的队尾元素**。

从队列中删除元素称为**出队**，元素出队后，其**后继元素**就成为**队首元素**。

队列具有**先进先出** (Fist In Fist Out，缩写为**FIFO**)的特性

对比:栈具有**先进后出**的特点(**FILO**)

```

1  typedef struct Node{
2      QueueElementType data;          /*数据域*/
3      struct Node *next;              /*指针域*/
4  }LinkQueueNode;
5
6  typedef struct {
7      LinkQueueNode *front;
8      LinkQueueNode *rear;
9  }LinkQueue;

```

3.2.2 队列的基本操作

```

1  int InitQueue(LinkQueue * Q) { /* 将Q初始化为一个空的链队列 */
2      Q->front = (LinkQueueNode *)malloc(sizeof(LinkQueueNode));
3      if(Q->front!=NULL){
4          Q->rear = Q->front;
5          Q->front->next = NULL;
6          return(TRUE);
7      }
8      else
9          return(FALSE);          /* 溢出! */
10 int EnterQueue(LinkQueue *Q, QueueElementType x){ /* 将数据元素x插入到队列Q中
    */
11     LinkQueueNode * NewNode;
12     NewNode=(LinkQueueNode * )malloc(sizeof(LinkQueueNode));
13     if(NewNode!=NULL) {
14         NewNode->data=x;
15         NewNode->next=NULL;
16         Q->rear->next=NewNode;
17         Q->rear=NewNode;
18         return(TRUE);
19     }
20     else return(FALSE);          /* 溢出! */
21 }
22 int DeleteQueue(LinkQueue * Q, QueueElementType *x) {
23     LinkQueueNode * p;
24     if(Q->front==Q->rear)
25         return(FALSE);
26     p=Q->front->next;
27     Q->front->next=p->next; /* 队头元素p出队 */
28     if(Q->rear==p) /* 如果队中只有一个元素p, 则p出队后成为空队 */
29         Q->rear=Q->front;
30     *x=p->data;
31     free(p); /* 释放存储空间 */
32     return(TRUE);
33 }

```

在数据结构代码的书写过程中,要注意释放存储空间!

3.2.3 特殊队列--循环队列

```
1  #define MAXSIZE 50  /*队列的最大长度*/
2  typedef struct{
3      QueueElementType  element[MAXSIZE];
4      /* 队列的元素空间*/
5      int  front;  /*头指针指示器*/
6      int  rear ;  /*尾指针指示器*/
7  }SeqQueue;·
```

把数组的**前端**和**后端**连接起来，形成一个**环形的顺序表**，即把存储队列元素的表从逻辑上看成一个环，称为**循环队列**或**环形队列**。

队空和队满的情况:rear == front

4 串

串的**逻辑结构**和**线性表**极为相似，区别仅在于**串的数据对象约束为字符集**。

串的**基本操作**和**线性表**有很大差别：

在线性表的基本操作中，大多以“**单个元素**”作为操作对象；

在串的基本操作中，通常以“**串的整体**”作为操作对象。

4.1 串的模式匹配

Brute Force算法

```
1  /*求从主串s的下标pos起，串t第一次出现的位置，成功返回位置序号，不成功返回-1*/
2  int StrIndex(SString s,int pos, SString t) {
3      int i, j, start;
4      if (t.len==0)
5          return(0);          /* 模式串为空串时，是任意串的匹配串 */
6      start=pos;
7      i=start;
8      j=0;                    /* 主串从pos开始，模式串从头（0）开始 */
9      while (i<s.len && j<t.len)
10         if (s.ch[i]==t.ch[j]) {
11             i++;
12             j++;
13         }                    /* 当前对应字符相等时推进 */
14         else {
15             start++;          /* 当前对应字符不等时回溯 */
16             i=start;
17             j=0;              /* 主串从start+1开始，模式串从头（0）开始*/
18         }
19         if (j>=t.len)
20             return(start);    /* 匹配成功时，返回匹配起始位置 */
21         else
22             return(-1);       /* 匹配不成功时，返回-1 */
23 }
```

4.2 KMP算法

[B站的KMP算法讲解\(甚至是学姐讲课\)](#)

5 数组与广义表

5.1 数组

5.1.1 计算数组元素的地址

例题:对一个二维数组 A'[6][8]' 按行优先存储, 若 A'[0][0]' 的地址为1000, 每个元素占4字节, 则 A'[3][5]' 的地址是1124.

5.1.2 稀疏矩阵三元组表表示

```
1  #define MAXSIZE 1000          /*非零元素的个数最多为1000*/
2  typedef struct{
3      int row, col;              /*该非零元素的行下标和列下标*/
4      ElementType e;             /*该非零元素的值*/
5  }Triple;
6  typedef struct{
7      Triple data[MAXSIZE];      /* 非零元素的三元组表 */
8      int m, n, len;             /*矩阵的行数、列数和非零元素的个数*/
9  }TSMatrix;
```

5.1.3 稀疏矩阵转置

```
1  /*把矩阵A转置到B所指向的矩阵中去。矩阵用三元组表表示*/
2  void TransposeTSMatrix(TSMatrix A, TSMatrix * B) {
3      int i, j, k;
4      B->m= A.n ; B->n= A.m ; B->len= A.len ;
5      if(B->len>0) {
6          j=0;                    /* j为三元组表B的下标 */
7          for(k=0; k<A.n; k++)    /* 扫描三元组表A共n次 */
8              for(i=0; i<A.len; i++) /* i为三元组表A的下标 */
9                  if(A.data[i].col==k){/* 寻找三元组表A的列值为k的进行转置 */
10                     B->data[j].row=A.data[i].col;
11                     B->data[j].col=A.data[i].row;
12                     B->data[j].e=A.data[i].e;
13                     j++;
14                 }                /* 内循环if结束*/
15     }                            /* if(B->len>0)结束*/
16 }
```

上述方法需要两次遍历,时间复杂度较高.

以下是一个快速的转置方法:

```

1 FastTransposeTSMatrix (TSMatrix A, TSMatrix * B) {
2     int col , t , p, q;
3     int num[MAXSIZE], position[MAXSIZE] ;
4     B->len= A.len ; B->n= A.m ; B->m= A.n ;
5     if(B->len) {
6         for(col=0;col<A.n;col++)
7             num[col]=0;          /*清零num数组*/
8         for(t=0;t<A.len;t++)
9             num[A.data[t].col]++; /*计算三元组表A每一列的非零元素的个数*/
10        position[0]=0;
11        for(col=1;col<A.n;col++) /*求col列中第一个非零元素在B.data[ ]中的正确位置*/
12            position[col]=position[col-1]+num[col-1];
13        for(p=0;p<A.len;p++) { /* 从头扫描三元组表A一次 */
14            col=A.data[p].col;
15            q=position[col]; /*col列中第一个非零元素在B.data[ ]中的正确位置*/
16            B->data[q].row=A.data[p].col;
17            B->data[q].col=A.data[p].row;
18            B->data[q].e=A.data[p].e;
19            position[col]++; /*col列中下一个非零元素在B.data[ ]中的正确位置, 修改了position数组*/
20        }
21    }
22 }

```

这个转置方法原理如下:

$$\mathbf{A} = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

上图中的矩阵由A转置为B,我们需要建立两个数组num和position.

num数组存储每一列元素的个数,position数组根据num数组计算每一列第一个元素转置后在data数组的位置

每一列的元素进入data数组后,position数组中的元素自增.

col	0	1	2	3	4	5	6
num[col]	2	2	2	1	0	1	0
position[col]	0	2	4	6	7	7	8

5.2 十字链表

5.2.1 结构定义

```

1  typedef struct OLNode{
2      int row, col;           /* 非零元素的行和列下标 */
3      ElementType value;
4      struct OLNode *down, *right; /* 非零元素所在行表列表的后继链域 */
5  }OLNode; *OLink;
6
7  typedef struct {
8      OLink * row_head, *col_head; /* 行、列链表的头指针向量 */
9      int m, n, len;              /* 稀疏矩阵的行数、列数、非零元素的个数 */
10 }CrossList;

```

5.2.2 创建十字链表

```

1  /*采用十字链表存储结构，创建稀疏矩阵M*/
2  CreateCrossList (CrossList * M) {
3      int m,n,t,i,j;
4      OLink p,q;
5      printf("输入M的行数、列数和非零元素的个数\n");
6      scanf("%d,%d,%d",&m,&n,&t); /*输入M的行数、列数和非零元素的个数*/
7      M->m=m; M->n=n; M->len=t;
8      if(!(M->row_head=(OLink *)malloc(m*sizeof(OLink))))
9          exit(OVERFLOW);
10     if(!(M->col_head=(OLink *)malloc(n*sizeof(OLink))))
11         exit(OVERFLOW);
12     for(i=0,i<m,i++)
13         M->row_head[i]=NULL; /*初始化行头指针向量*/
14     for(j=0,j<n,j++)
15         M->col_head[j]=NULL; /*初始化列头指针向量*/
16     for(scanf(&i,&j,&e);i!=-1;scanf(&i,&j,&e)) {
17         if(!(p=(OLNode *)malloc(sizeof(OLNode))))
18             exit(OVERFLOW);
19         p->row=i; p->col=j; p->value=e; /*生成结点*/
20         if(M->row_head[i]==NULL)
21             M->row_head[i]=p;
22         else{ /*寻找行表中的插入位置*/
23             q=M->row_head[i];
24             while(q->right&&q->right->col<j)
25                 q=q->right; /*找到链表末尾插入*/
26         }
27         p->right=q->right;

```

```

28     q->right=p;      /*完成插入*/
29 }
30 if(M->col_head[j]==NULL)
31     M->col_head[j]=p;
32 else{ /*寻找列表中的插入位置*/
33     q=M->col_head[j];
34     while(q->down&&q->down->row<i);
35     q=q->down; /*找到链表末尾插入*/
36     p->down=q->down;
37     q->down=p;      /*完成插入*/
38 }
39 } /*for结束*/
40 }

```

5.3 广义表

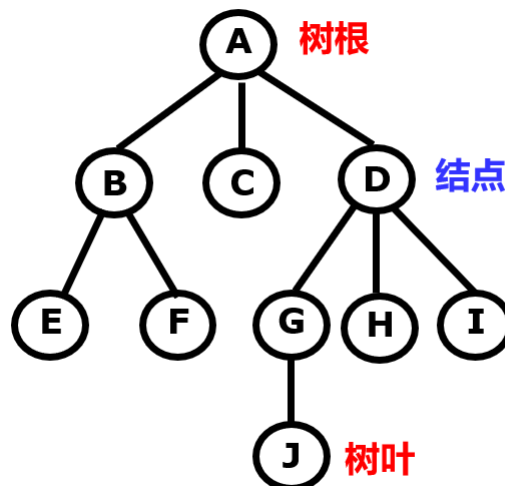
```

1  typedef enum {ATOM, LIST} ElemTag; /*ATOM=0, 表示原子; LIST=1, 表示子表*/
2  typedef struct GLNode {
3      ElemTag tag; /*标志位tag用来区别原子结点和表结点*/
4      union {
5          AtomType atom; /*原子结点的值域atom*/
6          struct {
7              struct GLNode * hp, *tp;
8          } htp; /*表结点的指针域htp, 包括表头指针域hp和表尾指针域tp*/
9      } atom_htp; /* atom_htp 是原子结点的值域atom和表结点的指针域htp的联合体域*/
10 } GLNode, *GList;

```

6 树与二叉树

6.1 树的定义



树描述的是一种**层次结构**,元素间是**一对多**的逻辑关系

6.2 二叉树的链式存储结构

```
1 typedef struct Node {
2     ElemType data;           // 数据域
3     struct Node *lchild, *rchild; // 指针域
4 } BiNode, *BiTree;
```

6.3 二叉树的遍历

6.3.1 先序遍历DLR算法

```
1 /*先序遍历二叉树，root为指向二叉树(或某一子树)根结点的指针*/
2 void PreOrder(BiTree root) {
3     if (root!=NULL) {
4         Visit(root->data);           /*访问根结点*/
5         PreOrder(root->Lchild);       /*先序遍历左子树*/
6         PreOrder(root->Rchild);       /*先序遍历右子树*/
7     }
8 }
```

6.3.2 中序遍历LDR算法

```
1 /*中序遍历二叉树，root为指向二叉树(或某一子树)根结点的指针*/
2 void InOrder(BiTree root) {
3     if (root!=NULL) {
4         InOrder(root->Lchild);       /*中序遍历左子树*/
5         Visit(root->data);           /*访问根结点*/
6         InOrder(root->Rchild);       /*中序遍历右子树*/
7     }
8 }
```

6.3.3 后序遍历LRD算法

```
1 /* 后序遍历二叉树，root为指向二叉树(或某一子树)根结点的指针*/
2 void PostOrder(BiTree root) {
3     if (root!=NULL) {
4         PostOrder(root->Lchild);     /*后序遍历左子树*/
5         PostOrder(root->Rchild);     /*后序遍历右子树*/
6         Visit(root->data);           /*访问根结点*/
7     }
8 }
```

6.3.4 先序遍历创建二叉链表

```
1 void CreateBiTree(BiTree *bt) {
2     char ch;
3     ch = getchar();
4     if(ch=='.') *bt=NULL;
5     else {
6         *bt=(BiTree)malloc(sizeof(BiTNode));    //生成一个新结点
7         (*bt)->data=ch;
8         CreateBiTree(&((*bt)->LChild));          //生成左子树
9         CreateBiTree(&((*bt)->RChild));          //生成右子树
10    }
11 }
```

6.3.5 二叉树的层次遍历

```
1 void LevelOrder(BiTree bt){
2     BiTree Queue[MAXNODE];                /*定义队列*/
3     int front, rear;
4     if (bt==NULL) return;                 /*空二叉树，遍历结束*/
5     front=0; rear=0;
6     Queue[rear]=bt;                       /*根结点入队列*/
7     rear++;
8     while((rear!=front)){                 /*队列不空，继续遍历，否则，遍历结
束*/
9         visit(Queue[front]->data);        /*访问刚出队的元素*/
10        if (Queue[front]->lchild!=NULL) {  /*如果有左孩子，左孩子入队*/
11            Queue[rear]=Queue[front]->lchild;
12            rear++;
13        }
14        if (Queue[front]->rchild!=NULL){   /*如果有右孩子，右孩子入队*/
15            Queue[rear]=Queue[front]->rchild;
16            rear++;
17        }
18        front++;                           /*出队*/
19    }
20 }
```

6.3.6 非递归先序遍历

```
1 void PreOrder1(BiTNode *b) {
2     BiTNode *p;
3     SqStack *st;                          //定义栈指针st
4     InitStack(st);                       //初始化栈st
5     if (b!=NULL) {
6         Push(st, b);                     //根结点进栈
7         while (!StackEmpty(st)) {        //栈不为空时循环
8             Pop(st, p);                   //退栈结点p并访问它
9             printf("%c ", p->data);
10            if (p->rchild!=NULL)           //有右孩子时将其进栈
```

```

11         Push(st, p->rchild);
12         if (p->lchild!=NULL) //有左孩子时将其进栈
13             Push(st, p->lchild);
14     }
15     printf("\n");
16 }
17 DestroyStack(st); //销毁栈
18 }

```

```

1 void PreOrder2(BitNode *b){
2     BitNode *p; SqStack *st; //定义一个顺序栈指针st
3     InitStack(st); //初始化栈st
4     p=b;
5     while (!StackEmpty(st) || p!=NULL) {
6         while (p!=NULL) { //访问结点p及其所有左下结点并进栈
7             printf("%c ", p->data); //访问结点p
8             Push(st, p); //结点p进栈
9             p=p->lchild; //移动到左孩子
10        }
11        //以下考虑栈顶结点
12        if (!StackEmpty(st)) { //若栈不空
13            Pop(st, p); //出栈结点p
14            p=p->rchild; //转向处理其右子树
15        }
16    }
17    printf("\n");
18    DestroyStack(st); //销毁栈
19 }

```

6.3.7 非递归中序遍历

```

1 void InOrder (BiTree root) { //中序遍历二叉树的非递归算法
2     InitStack (&S);
3     p=root;
4     while(p!=NULL || !IsEmpty(S)){
5         if (p!=NULL){ //根指针进栈，遍历左子树
6             Push(&S,p);
7             p=p->Lchild;
8         }
9         else{ //根指针退栈，访问根结点，遍历右子树
10            Pop(&S,&p);
11            Visit(p->data);
12            p=p->Rchild;
13        }
14    }
15 }

```


6.3.8 非递归后序遍历

```
1 void PostOrder1(BiTNode *b) { //后序非递归遍历算法
2     BiTNode *p, *r=NULL;
3     SqStack *st; //定义一个顺序栈指针st
4     InitStack(st); //初始化栈st
5     p=b;
6     while (p!=NULL || !IsEmpty(st)) {
7         if(p!=NULL) {
8             Push(st, p); //结点p进栈
9             p=p->lchild; //移动到左孩子(遍历左子树)
10        }
11        else {
12            GetTop(st, &p); //取出当前的栈顶结点到p中 (p
指向它)
13            if (p->rchild== NULL || p->rchild ==r) { //若结点p的右孩子为空或者为刚
访问结点
14                printf("%c ", p->data); //访问结点p
15                r=p; //r指向刚访问过的结点
16                Pop(st, &p);
17                p=NULL;
18            }
19            else
20                p=p->rchild; //转向处理其右子树
21        }
22    }
23    printf("\n");
24    DestroyStack(st); //销毁栈
25 }
```

总结:非递归遍历需要用到栈,三种遍历方式的差别在于访问节点的顺序.

DLR算法1是节点出栈时访问,并且将孩子节点入栈;

DLR算法2是直接所有的左孩子访问并入栈;

LDR是左孩子进栈,出栈,访问,右孩子入栈;

LRD将访问放到最后,从左往右,若不存在孩子结点则访问.

6.3.9 由遍历反推二叉树

注意:一定要有中序遍历!

6.4 特殊的树

6.4.1 哈夫曼树

```
1 void CreateHT(HTNode ht[],int n) {           //构造哈夫曼树
2     int i,k,lnode,rnode;
3     double min1,min2;
4     for (i=0;i<2*n-1;i++)
5         ht[i].parent=ht[i].lchild=ht[i].rchild= -1;    //所有节点的域置初值-1
6     for (i=n;i<2*n-1;i++) {                   //构造哈夫曼树的n-1个节点
7         min1=min2=32767;                       //lnode和rnode为最小权重的两个节点位置
8         lnode=rnode=-1;
9         for (k=0;k<=i-1;k++)                  //在ht[0..i-1]中找权值最小的两个节点
10            if (ht[k].parent== -1) {           //只在尚未构造二叉树的节点中查找
11                if (ht[k].weight<=min1) {
12                    min2=min1;    rnode=lnode;
13                    min1=ht[k].weight;    lnode=k;
14                }
15                else if (ht[k].weight<=min2) {
16                    min2=ht[k].weight;    rnode=k;
17                }
18            }
19            ht[i].weight=ht[lnode].weight+ht[rnode].weight;    //ht[i]作为双亲节点
20            ht[i].lchild=lnode;ht[i].rchild=rnode;
21            ht[lnode].parent=i;ht[rnode].parent=i;
22        }
23    }
```

6.4.2 双亲表示法定义

```
1 #define MAX 100
2 typedef struct TNode{
3     DataType data;
4     int parent;
5 } TNode;
6 typedef struct {
7     TNode tree[MAX];
8     int nodenum; /*结点数*/
9 } ParentTree;
```

6.4.3 孩子表示法定义

```
1 typedef struct ChildNode { /* 孩子链表结点的定义 */
2     int Child;           /* 该孩子结点在线性表中的位置 */
3     struct ChildNode * next; /* 指向下一个孩子结点的指针 */
4 } ChildNode;
5 typedef struct { /* 顺序表结点的结构定义 */
6     DataType data;      /* 结点的信息 */
7     ChildNode * FirstChild; /* 指向孩子链表的头指针 */
8 } DataNode;
9 typedef struct { /* 树的定义 */
10     DataNode nodes[MAX]; /* 顺序表 */
11     int root,num;        /* 该树的根结点在线性表中的位置和该树的结点个数 */
12 } ChildTree;
```

6.4.4 孩子兄弟表示法定义

```
1 typedef struct node{
2     ElemType data;
3     struct node * firstchild, * nextsibling;
4 } Tnode, *Ptree;
```

7 图

7.1 图的存储方式

7.1.1 图的邻接矩阵

定义

```
1 #define MAX_VERTEX_NUM 20 /*最多顶点个数*/
2 #define INFINITY 32768 /*表示极大值∞*/
3 typedef enum {DG, DN, UDG, UDN} GraphKind;
4 /*图的种类: DG表示有向图, DN表示有向网, UDG表示无向图, UDN表示无向网*/
5 typedef char vertexData; /*假设顶点数据为字符型*/
6 typedef struct ArcNode {
7     AdjType adj; /*对于无权图, 用1或0表示是否相邻; 对带权图, 则为权值类型*/
8 } ArcNode;
9 typedef struct{
10     vertexData vertex[MAX_VERTEX_NUM]; /*顶点向量*/
11     ArcNode arcs [MAX_VERTEX_NUM][MAX_VERTEX_NUM]; /*邻接矩阵*/
12     int vexnum, arcnum; /*图的顶点数和弧数*/
13     GraphKind kind; /*图的种类标志*/
14 } AdjMatrix;
```

创建有向图

```
1 int LocateVertex(AdjMatrix * G, vertexData v) { /*求顶点位置函数*/
2     int j=false,k;
```

```

3     for(k=0;k<G->vexnum;k++)
4         if(G->vertex[k]==v) {
5             j=k;
6             break;
7         }
8     return j;
9 }
10 int CreatedN(AdjMatrix *G) {    /*创建一个有向网*/
11     int i,j,k,weight;
12     VertexData v1,v2;
13     scanf("%d,%d",&G->arcnum,&G->vexnum);    /*输入图的顶点数和弧数*/
14     for(i=0;i<G->vexnum;i++)
15         for(j=0;j<G->vexnum;j++)
16             G->arcs[i][j].adj=INFINITY;    /*初始化邻接矩阵*/
17     for(i=0;i<G->vexnum;i++)
18         scanf("%c",&G->vertex[i]);    /* 输入图的顶点*/
19     for(k=0;k<G->arcnum;k++) {
20         scanf("%c,%c,%d",&v1,&v2,&weight);    /*输入一条弧的两个顶点及权值*/
21         i=LocateVex_M(G,v1);
22         j=LocateVex_M(G,v2);
23         G->arcs[i][j].adj=weight;    /*建立弧*/
24     }
25     return true ;
26 }

```

7.1.2 有向图

```

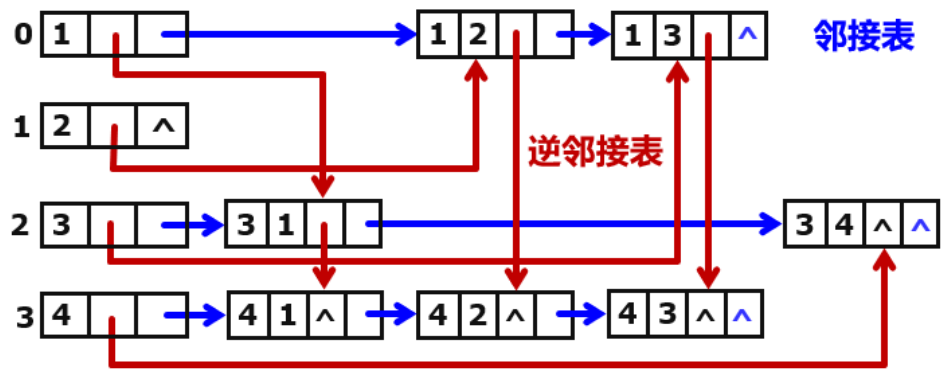
1  #define  MAX_VERTEX_NUM  20    /*最多顶点个数*/
2  typedef  enum{DG, DN, UDG, UDN}  GraphKind;    /*图的种类*/
3  typedef  struct  ArcNode  {
4      int  adjvex;    /*该弧指向顶点的位置*/
5      struct  ArcNode  *nextarc;    /*指向下一条弧的指针*/
6      OtherInfo  info;    /*与该弧相关的信息*/
7  } ArcNode;
8  typedef  struct  VertexNode  {
9      VertexData  data;    /*顶点数据*/
10     ArcNode  *firstarc;    /*指向该顶点第一条弧的指针*/
11 } VertexNode;
12 typedef  struct  {
13     VertexNode  vertex[MAX_VERTEX_NUM];
14     int  vexnum, arcnum;    /*图的顶点数和弧数*/
15     GraphKind  kind;    /*图的种类标志*/
16 }AdjList;

```

储存出度的图是邻接表,存储入度的图是逆邻接表

7.1.3 十字链表

十字链表是邻接表和逆邻接表的结合



定义:

```
1  #define  MAX_VERTEX_NUM    20                /*最多顶点个数*/
2  typedef  enum{DG, DN, UDG, UDN}  GraphKind; /*图的种类*/
3  typedef  struct  ArcNode  {
4      int  headvex, tailvex;                // 头尾编号
5      struct  ArcNode  *hlink, *tlink;    // 链域
6  } ArcNode;
7  typedef  struct  VertexNode  {
8      ElemType  data;                    // 数据域（顶点信息）
9      ArcNode  *firstin, *firstout;      // 边表头指针
10 } VertexNode;
11 typedef  struct  {
12     VertexNode  vertex[MAX_VERTEX_NUM];
13     int  vexnum, arcnum;                /*图的顶点数和弧数*/
14     GraphKind  kind;                    /*图的种类标志*/
15 } OrthList;
```

头结点类型

<i>data</i>	<i>firstin</i>	<i>firstout</i>
顶点信息	以该顶点为弧头的第一个弧结点	以该顶点为弧尾的第一个弧结点

边结点类型

<i>tailvex</i>	<i>headvex</i>	<i>hlink</i>	<i>tlink</i>	<i>info</i>
弧头编号	弧尾编号	指向弧头相同的下一条弧	指向弧尾相同的下一条弧	弧的权等信息

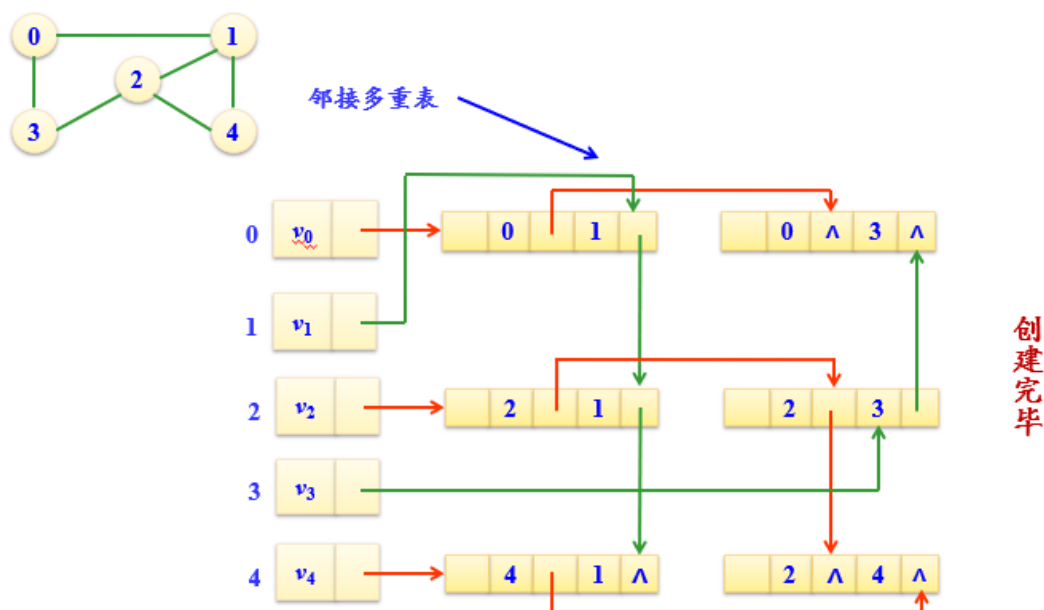
7.1.4 邻接多重表

头结点类型

<i>data</i>	<i>firstedge</i>
顶点信息	第一条依附该顶点的边

边结点类型

<i>mark</i>	<i>ivex</i>	<i>ilink</i>	<i>jvex</i>	<i>jlink</i>	<i>info</i>
标志域可以标记该边是否被搜索过	边的顶点 <i>i</i>	下一条依附于顶点 <i>i</i> 的边结点	边的顶点 <i>j</i>	下一条依附于顶点 <i>j</i> 的边结点	边的权



7.2 图的遍历

7.2.1 DFS深度优先搜索

- 1) 从图中某个顶点 v_0 出发,首先访问 v_0 .
- 2) 依次以 v_0 的未被访问的邻接点为出发点,深度优先搜索图,直至图中所有与 v_0 有路径相通的顶点都被访问.

核心函数需求:

设置一个**标志数组** `visited[1..n]`

1.初始化: `visited[i] = 0` ($i \in [0, n-1]$)

2.若顶点 w 被访问,则令`visited[w]=1`

还需要能够求得**当前结点的邻接点**

1.求初始结点 v_0 的第一个邻接点

2.求当前结点 w 的下一个邻接点

非递归实现深度优先搜索:

```

1 void DepthFirstSearch1(Graph g, int v0) {
2     InitStack(S); /*初始化空栈*/
3     Push(S, v0);
4     while (!Empty(S)) {
5         v = Pop(S);
6         if (!visited(v)) { /*栈中可能有重复结点*/
7             visit(v);
8             visited[v] = True;
9         }
10        w = FirstAdjVertex(g, v); /*求v的第一个邻接点*/
11        while (w != -1) {
12            if (!visited(w)) Push(S, w);
13            w = NextAdjVertex(g, v, w); /*求v相对于w的下一个邻接点*/
14        }
15    }
16 }

```

```

15     }
16 }

```

```

1 void DepthFirstSearch2(Graph G, int v0) {
2     InitStack(&S); //初始化空栈
3     visit(v0); //访问最初的结点
4     visited[v0]=true; //标记初结点已被访问
5     Push(&S,v0); //初结点入栈
6     while(!IsEmpty(&S)) { //当栈内存在元素时
7         GetTop(&S,&v); //取出栈顶元素
8         w=FirstAdjVertex(G,v); //w存储与v结点最近的相邻结点
9         while(w!=-1) { //当无最近结点时FAV函数返回-1
10             if(!visited[w]) { //w结点未被访问过时
11                 visit(w);
12                 visited[w]=true;
13                 Push(&S,w);
14                 break;
15             } //访问w结点并标记,回到循环,此时w结点变为下一次的v结点
16             else{
17                 w=NextAdjVertex(G,v,w); //否则就进入下一个与v结点相邻的结点
18             }
19             if(w== -1) Pop(&S,v); //无相邻结点,退回上一级结点
20         }
21     }

```

7.2.2 BFS广度优先算法

广度优先搜索 (Breadth First Search) 是指按照广度方向搜索, 它类似于**树的层次遍历**.

访问某个起始顶点 V_0 , 将 V_0 作为当前顶点.

依次访问**当前顶点的所有未访问过的邻接点**.

然后分别从这些邻接顶点出发**广度优先遍历图**.

直至图中所有**已被访问过的顶点的邻接点**都已被访问过为止.

代码实现:

```

1 void BreadthFirstSearch(Graph g,int v0) { /*广度优先搜索图g中v0所在的连通子图*/
2     visit(v0);
3     visited[v0]=True;
4     InitQueue(&Q); /*初始化空队*/
5     EnterQueue(&Q,v0); /* v0进队*/
6     while(!Empty(Q)) {
7         DeleteQueue(&Q,&v); /*队头元素出队*/
8         w=FirstAdjVertex(g,v); /*求v的第一个邻接点*/
9         while(w!=-1) {
10             if(!visited(w)) {
11                 visit(w);
12                 visited[w]=True;
13                 EnterQueue(&Q, w);
14             }

```

```

15         w=NextAdjVertex(g,v,w);    /*求v相对于w的下一个邻接点*/
16     }
17 }
18 }

```

7.3 图的应用

7.3.1 求简单路径

```

1  int pre[];    //pre数组储存该节点前驱地址
2  void one_path(Graph *G, int u, int v) {
3      /*在连通图G中找一条从第u个顶点到v个顶点的简单路径*/
4      int i;
5      pre=(int *)malloc(G->vexnum*sizeof(int));
6      for(i=0;i<G->vexnum;i++) /* 初始化pre数组的各个元素为不可能的下标值-1*/
7          pre[i]=-1;
8      pre[u]=-2;                /*将初始顶点u的pre[u]置为-2，表示初始顶点u已被访问，且没前驱*/
9      DFS_path(G, u, v);    /*用深度优先搜索找一条从u到v的简单路径。*/
10     free(pre);    //释放pre数组空间
11 }
12 int DFS_path(Graph *G, int u, int v) {
13     /*在连通图G中用深度优先搜索策略找一条从u到v的简单路径。*/
14     int j;
15     for(j=firstadj(G,u);j>=0;j=nextadj(G,u,j)){ //j初始值赋为u的最近结点,每次j调整
        整为u的下一个临近结点
16         if(pre[j]==-1) {
17             pre[j]=u; //若j没有前驱,则将j前驱赋值为u
18             if(j==v){ //此时已经有一条u到v的路径
19                 print_path(pre,v); /*从v开始沿着pre[]中保留的前驱指针输出路径u到v*/
20                 return 1;
21             }
22             else if(DFS_path(G,j,v)){
23                 return 1;
24             }
25         }
26     }
27     return 0;
28 }

```

现在拿一个简单图来解析一下里面的难点:

图 1-0-2-3-4 以0作为起点,4作为终点,

0的第一个邻接结点是1,此时 `pre[1] == -1`,进入第一个if语句,

将1前驱变为0,此时不满足if和else if分支,跳出此层,

j在for循环中自增变为0的第二个邻接结点2,

2满足if语句,递归搜索到4.

7.3.2 最小生成树

7.3.2.1 Prim算法

设：图 $G=(V,E)$ ，生成树 T 的顶点集合为 U

①任取图中一个顶点 v_0 加入集合 U

②在所有满足 $u \in U, v \in V-U$ 的边 $(u,v) \in E$ 中

找到一个具有**最小值的边**加入生成树 T 中

并将与之邻接的顶点 v 加入集合 U

③若全部 n 个顶点均已加入到 U 集合中，则算法结束

1 | 否则继续执行第2步

```
1  #define INF 32767          //INF表示∞
2  void Prim(MatGraph g, int v) {
3      int lowcost[MAXV];
4      int min;
5      int closest[MAXV], i, j, k;
6      for (i=0; i<g.n; i++) {          //给lowcost[]和closest[]置初值
7          lowcost[i]=g.edges[v][i];
8          closest[i]=v;
9      }
10     for (i=1; i<g.n; i++) {          //输出(n-1)条边
11         min=INF;
12         for (j=0; j<g.n; j++)          //在(V-U)中找出离U最近的顶点k
13             if (lowcost[j]!=0 && lowcost[j]<min) {
14                 min=lowcost[j];
15                 k=j;          //k记录最近顶点编号
16             }
17         printf(" 边(%d, %d)权为:%d\n", closest[k], k, min);
18         lowcost[k]=0;          //标记k已经加入U
19         for (j=0; j<g.n; j++) //修改数组lowcost和closest
20             if (lowcost[j]!=0 && g.edges[k][j]<lowcost[j]) {
21                 lowcost[j]=g.edges[k][j];
22                 closest[j]=k;
23             }
24     } //外层for循环结束
25 }
```

在这段代码中最重要的就是**lowcost数组**和**closest数组**的取法

lowcost数组中存放到**V集合中到当前U集合中的最短边**.

closest数组存放与当前的lowcost数组**所对应的V集合中的结点**.

7.3.2.2 Kruskal算法

```
1 void kruskal(MatGraph g) {
2     int i, j, u1, v1, sn1, sn2, k;
3     int vset[MAXV];      //vset数组通过加边记录不同连通分量
4     Edge E[MaxSize];     //存放所有边
5     k=0;                 //E数组的下标从0开始计
6     for (i=0;i<g.n;i++){ //由g产生的边集E
7         for (j=0;j<g.n;j++){
8             if (g.edges[i][j]!=0 && g.edges[i][j]!=INF) {
9                 E[k].u=i; E[k].v=j; E[k].w=g.edges[i][j];
10                k++;
11            }
12        }
13    } //将图的不同边通过E数组记录
14    Sort(E, g.e);         //E数组按权值递增排序
15    for (i=0;i<g.n;i++) { //初始化辅助数组
16        vset[i]=i;
17    }
18    k=1; //k当前构造成生成树的第几条边, 初值为1
19    j=0; //E中边的下标, 初值为0
20    while (k<g.n) { //生成的边数小于n时循环
21        u1=E[j].u;v1=E[j].v; //取一条边的头尾顶点
22        sn1=vset[u1];
23        sn2=vset[v1]; //分别得到两个顶点所属的集合编号
24        if (sn1!=sn2) { //两顶点属于不同的集合
25            printf("(%d, %d):%d\n", u1, v1, E[j].w);
26            k++; //生成边数增1
27            for (i=0;i<g.n;i++){
28                if (vset[i]==sn2){ //集合编号为sn2的改为sn1
29                    vset[i]=sn1;
30                }
31            }
32            j++; //扫描下一条边
33        } //end while
34    } //end kruskal
```

Kruskal算法本质是“避圈法”,防止生成回路,

因此Kruskal算法引入**vset数组**来计算连通分量,防止出现回路

7.3.2.3 对比

本质上来说,Prim算法和Kruskal算法的差别在于边和结点:

Prim算法有点类似于dijkstra算法,本质是不断加入新结点进v数组;

Kruskal算法则是挑选最小边,重点在于边.

普里姆最小生成树算法	克鲁斯卡尔最小生成树算法
以连通为主	以最小代价边为主
选保证连通的代价最小的邻接边	选不形成回路的当前最小代价边
算法时间复杂度: $O(n^2)$	算法时间复杂度: $O(e \log_2 e)$
算法时间复杂度与边无关	算法时间复杂度与边相关
适合于求 边稠密网 的最小生成树	适合于求 边稀疏网 的最小生成树

7.3.3 最短路径

7.3.3.1 Dijkstra算法

```

1 void Dijkstra(MatGraph g, int v) {
2     int dist[MAXV], path[MAXV];
3     int s[MAXV];
4     int mindis, i, j, u;
5     for (i=0; i<g.n; i++) {
6         dist[i]=g.edges[v][i];    //距离初始化
7         s[i]=0;                  //s[]置空
8         if (g.edges[v][i]<INF)    //路径初始化
9             path[i]=v;          //顶点v到i有边时
10        else
11            path[i]=-1;          //顶点v到i没边时
12    }
13    s[v]=1;                      //源点v放入S中
14    for (i=0; i<g.n; i++) {      //循环n-1次
15        mindis=INF;
16        for (j=0; j<g.n; j++)
17            if (s[j]==0 && dist[j]<mindis) {
18                u=j;
19                mindis=dist[j];
20            }
21        s[u]=1;                  //顶点u加入S中
22        for (j=0; j<g.n; j++)    //修改不在s中的顶点的距离
23            if (s[j]==0)
24                if (g.edges[u][j]<INF && dist[u]+g.edges[u][j]<dist[j])
25                    {
26                        dist[j]=dist[u]+g.edges[u][j];
27                        path[j]=u;
28                    }
29    }
30    Dispath(dist, path, s, g.n, v);    //输出最短路径
31 }

```

Dijkstra算法最重要的就是**dist数组**和**s数组**,

s数组存放的是**已经计算过的结点**,

dist数组存放s数组中结点到数组外结点的距离的最小值,

首先,dist数组初始化是由最初的结点到各结点的距离,不可直达则为INF,

每次加入新结点后将dist[u]与dist[v]+w作比较,更新dist数组的值.

7.3.3.2 Floyd 算法

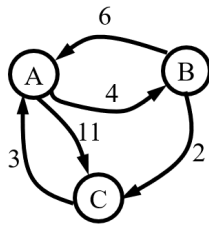
Floyd算法本质是对距离的一个递归更新,遍历方式是根据结点遍历整个矩阵.将每个结点均更新一遍就能找到最短距离.

```
1 void Floyd(MatGraph g) {           //求每对顶点之间的最短路径
2     int A[MAXVEX][MAXVEX]; //建立A数组
3     int path[MAXVEX][MAXVEX]; //建立path数组
4     int i, j, k;
5     for (i=0;i<g.n;i++) {
6         for (j=0;j<g.n;j++) {
7             A[i][j]=g.edges[i][j];
8             if (i!=j && g.edges[i][j]<INF){//i和j顶点之间有一条边时
9                 path[i][j]=i;
10            }
11            else{//i和j顶点之间没有一条边时
12                path[i][j]=-1;
13            }
14        }
15    }
16    for (k=0;k<g.n;k++) {           //对每个结点进行一个更新
17        for (i=0;i<g.n;i++){
18            for (j=0;j<g.n;j++){
19                if (A[i][j]>A[i][k]+A[k][j]) { //找到更短路径
20                    A[i][j]=A[i][k]+A[k][j]; //修改路径长度
21                    path[i][j]=path[k][j]; //修改最短路径为经过顶点
22                }
23            }
24        }
25    }
26 }
```

时间复杂度 $O(n^3)$

实例:

示例



初始状态: $\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$ 路径:

	AB	AC
BA		BC
CA		

在C、B之间加入A: $\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$ 路径:

	AB	AC
BA		BC
CA	CAB	

在A、C之间加入B: $\begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$ 路径:

	AB	ABC
BA		BC
CA	CAB	

在B、A之间加入C: $\begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$ 路径:

	AB	ABC
BCA		BC
CA	CAB	

7.3.4 拓扑排序

基于邻接表的拓扑排序(使用栈)

```

1  typedef struct ANode{
2      int adjvex;           //该边的终点编号
3      struct ANode *nextarc; //指向下一条边的指针
4      } ArcNode;
5
6  typedef struct {         //表头结点类型
7      vertex data;         //顶点信息
8      int count;           //存放顶点入度
9      ArcNode *firstarc;   //指向第一条边
10 } VNode;
11
12 typedef struct {
13     VNode adjlist[MAXV] ; //邻接表
14     int n, e;             //图中顶点数n和边数e
15 } AdjGraph;
16
17 void TopSort(AdjGraph *G) { //拓扑排序算法
18     int i, j;
19     int St[MAXV], top=-1;    //栈St的指针为top
20     ArcNode *p;
21     for (i=0;i<G->n;i++) {
22         G->adjlist[i].count=0;
23     } //入度置初值0
24     for (i=0;i<G->n;i++) { //求所有顶点的入度
25         p=G->adjlist[i].firstarc;
26         while (p!=NULL) {
27             G->adjlist[p->adjvex].count++;
28             p=p->nextarc;
29         }
30     }
31     for (i=0;i<G->n;i++) {
32         if (G->adjlist[i].count==0) {
33             top++;

```

```

34         St[top]=i;
35     }
36 } //将入度为0的顶点进栈
37
38 while (top>-1) { //栈不空循环
39     i=St[top];    top--; //出栈一个顶点i
40     printf("%d ",i); //输出该顶点
41     p=G->adjlist[i].firstarc; //找第一个邻接点
42     while (p!=NULL) { //将顶点i的出边邻接点的入度减1
43         j=p->adjvex;
44         G->adjlist[j].count--;
45         if (G->adjlist[j].count==0) { //将入度为0的邻接点进栈
46             top++;
47             St[top]=j;
48         }
49         p=p->nextarc; //找下一个邻接点
50     }
51 }
52 }

```